Energy Usage Forecasting Competition 2026

Abrao Soares Pereira and Mingjie Wei

Spring 2026

Setups

```
library(readxl)
library(lubridate)
library(ggplot2)
library(forecast)
library(Kendall)
library(tseries)
library(outliers)
library(tidyverse)
library(cowplot)
library(smooth)
library(forecast)
```

Load Datasets into R

```
# Load datasets from from repo as in excel

raw_load_data <- read_excel("~/Documents/ENVIRON 797 - TSA Spring 2026/Forecasting2026/data/load.xlsx")

raw_relativehumidity_data <- read_excel("~/Documents/ENVIRON 797 - TSA Spring 2026/Forecasting2026/data/

raw_temperature_data <- read_excel("~/Documents/ENVIRON 797 - TSA Spring 2026/Forecasting2026/data/tempe

submission_template <- read.csv("~/Documents/ENVIRON 797 - TSA Spring 2026/Forecasting2026/data/submissi


# Convert Excel datasets to CSV format
write_csv(raw_load_data,
          "~/Documents/ENVIRON 797 - TSA Spring 2026/Forecasting2026/data/load.csv")

write_csv(raw_relativehumidity_data,
          "~/Documents/ENVIRON 797 - TSA Spring 2026/Forecasting2026/data/relative_humidity.csv")

write_csv(raw_temperature_data,
          "~/Documents/ENVIRON 797 - TSA Spring 2026/Forecasting2026/data/temperature.csv")
```

Examine Datasets

```
# Process load - daily average
load_processed <- raw_load_data %>%
  mutate(
    date       = as.Date(date),
    daily_load = rowMeans(across(starts_with("h")), na.rm = TRUE)
  ) %>%
  select(date, daily_load) %>%
  arrange(date)

# Process temperature - average across 28 stations, then daily average
temp_cols <- grep("^t_ws", names(raw_temperature_data), value = TRUE)

temp_processed <- raw_temperature_data %>%
  mutate(
    date     = as.Date(date),
    avg_temp = rowMeans(across(all_of(temp_cols)), na.rm = TRUE)
  ) %>%
  filter(!is.na(date)) %>%
  group_by(date) %>%
  summarise(
    temp_mean = mean(avg_temp, na.rm = TRUE),
    temp_max  = max(avg_temp,  na.rm = TRUE),
    .groups   = "drop"
  )

# Process humidity - same two-step as temperature
rh_cols <- grep("^rh_ws", names(raw_relativehumidity_data), value = TRUE)

humid_processed <- raw_relativehumidity_data %>%
  mutate(
    date   = as.Date(date),
    avg_rh = rowMeans(across(all_of(rh_cols)), na.rm = TRUE)
  ) %>%
  group_by(date) %>%
  summarise(
    rh_mean = mean(avg_rh, na.rm = TRUE),
    .groups = "drop"
  )

head(load_processed)
```

```
## # A tibble: 6 x 2
##   date       daily_load
##   <date>          <dbl>
## 1 2005-01-01      3108.
## 2 2005-01-02      3068.
## 3 2005-01-03      3061.
## 4 2005-01-04      2566.
## 5 2005-01-05      2496.
## 6 2005-01-06      2735.
```

```r
head(temp_processed)
```

```
## # A tibble: 6 x 3
##   date       temp_mean temp_max
##   <date>         <dbl>    <dbl>
## 1 2005-01-01      53.6     68.2
## 2 2005-01-02      53.8     65.7
## 3 2005-01-03      55.9     67.3
## 4 2005-01-04      61.7     72
## 5 2005-01-05      60.4     68.6
## 6 2005-01-06      62.0     68.1
```

```r
head(humid_processed)
```

```
## # A tibble: 6 x 2
##   date       rh_mean
##   <date>       <dbl>
## 1 2005-01-01    76.7
## 2 2005-01-02    80.5
## 3 2005-01-03    81.2
## 4 2005-01-04    74.8
## 5 2005-01-05    76.1
## 6 2005-01-06    78.0
```

```r
summary(load_processed)
```

```
##       date              daily_load
##  Min.   :2005-01-01   Min.   :2247
##  1st Qu.:2006-08-16   1st Qu.:2798
##  Median :2008-03-31   Median :3506
##  Mean   :2008-03-31   Mean   :3629
##  3rd Qu.:2009-11-14   3rd Qu.:4211
##  Max.   :2011-06-30   Max.   :7897
```

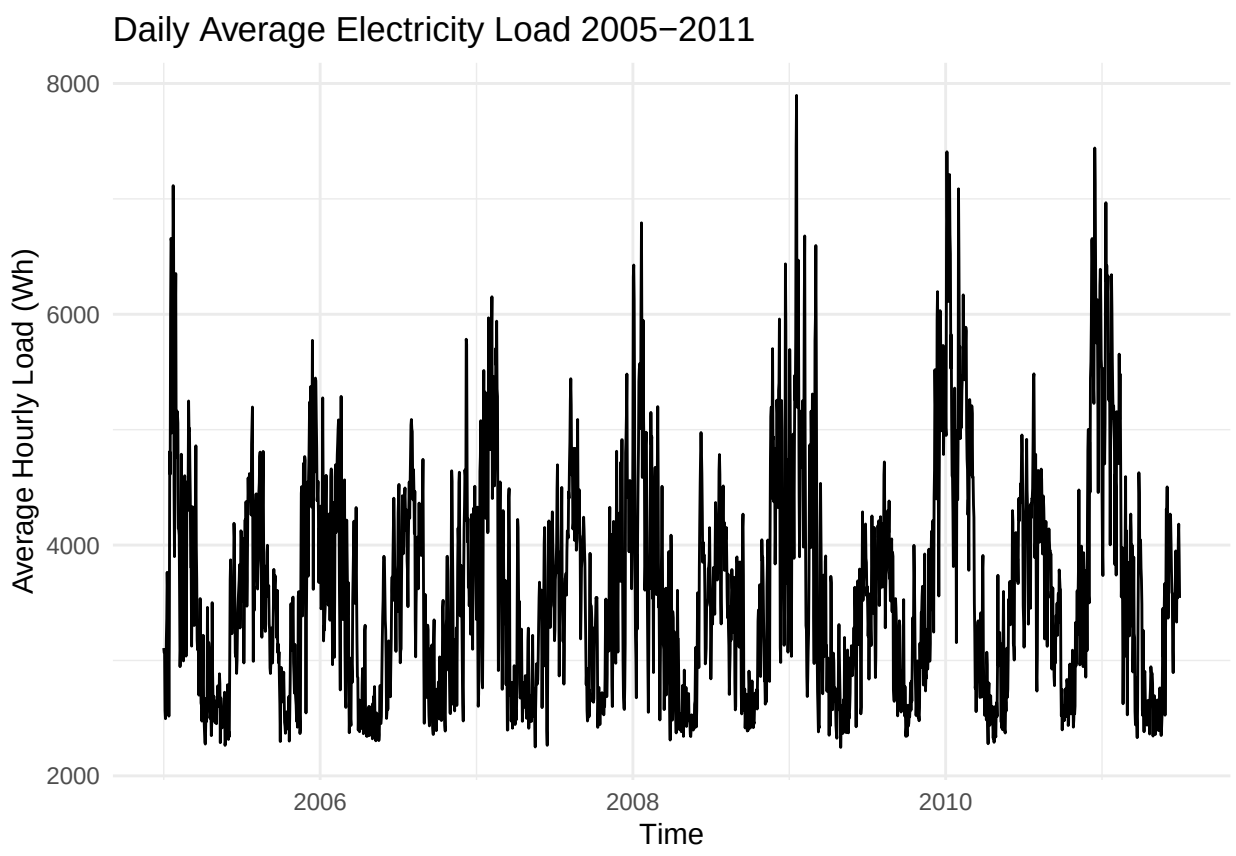
Plots and Figures

```r
ts_load <- msts(
  load_processed$daily_load,
  seasonal.periods = c(7, 365),
  start = c(2005, 1)
)
```

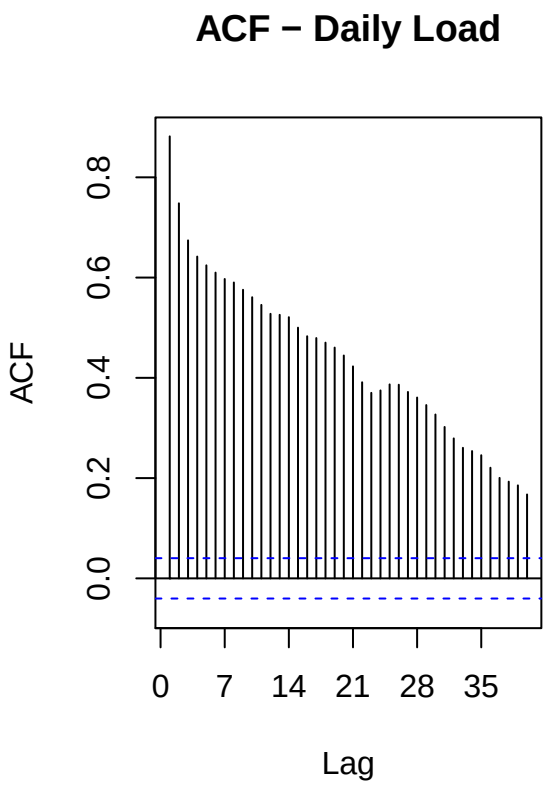
First Plot

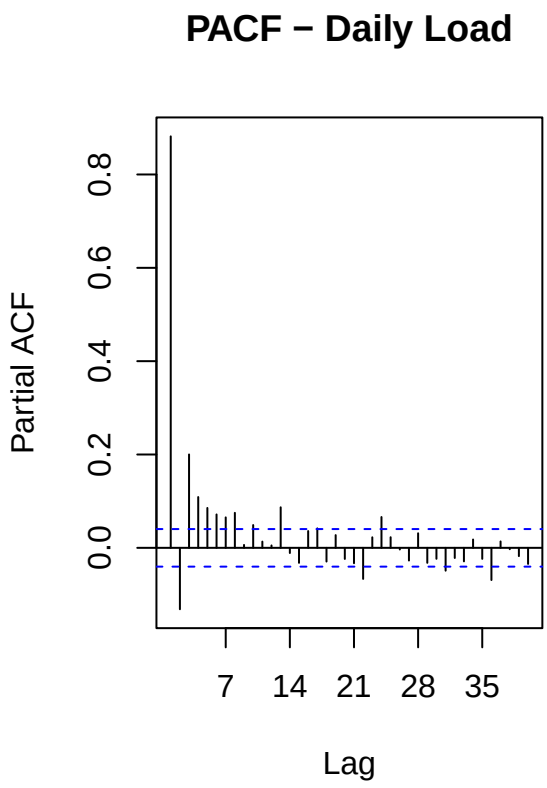
```r
# Time series plot
autoplot(ts_load) +
  labs(
```

```
    title = "Daily Average Electricity Load 2005-2011",
    x     = "Time",
    y     = "Average Hourly Load (Wh)"
  ) +
  theme_minimal()
```




```
# ACF and PACF
par(mfrow = c(1, 2))
Acf(ts_load,  lag = 40, plot = TRUE, main = "ACF - Daily Load")
Pacf(ts_load, lag = 40,              main = "PACF - Daily Load")
```



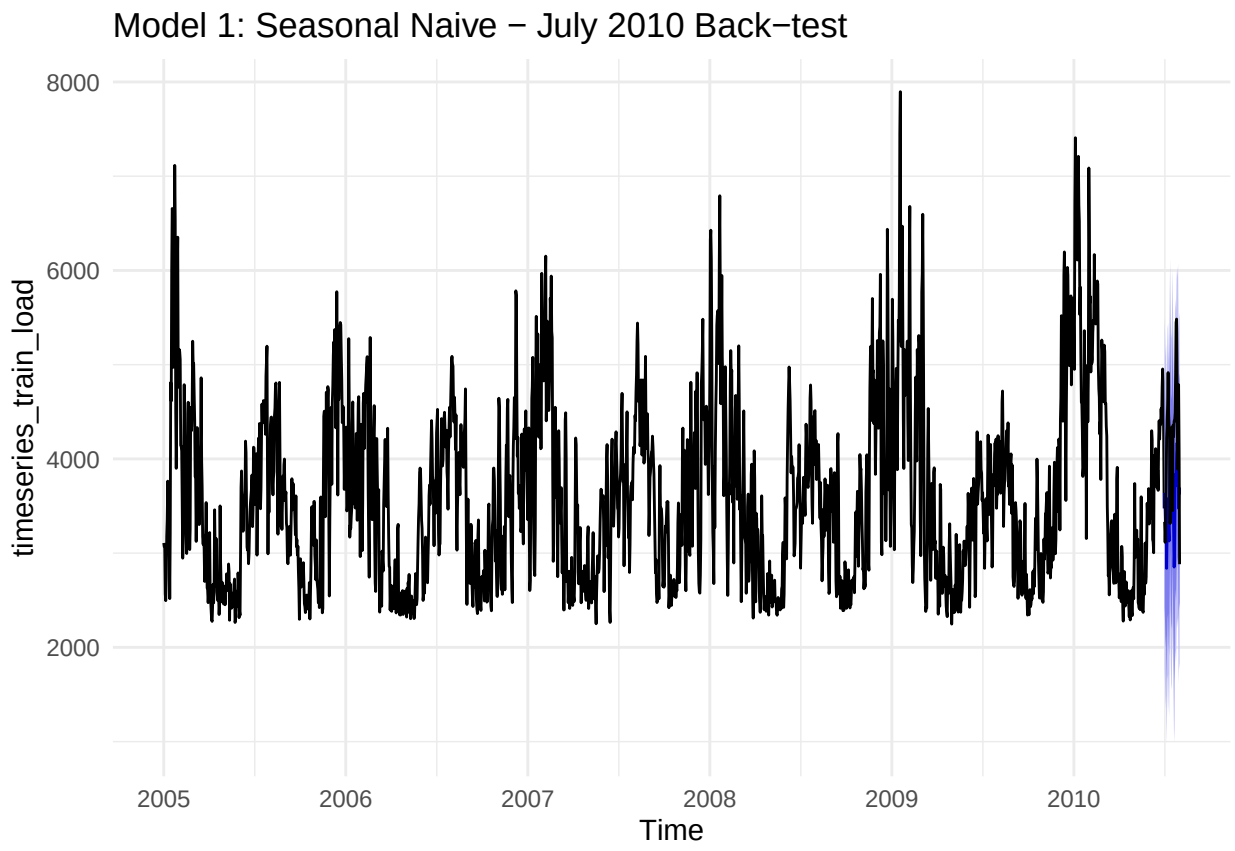



```
par(mfrow = c(1, 1))
```

Train Model and Testing

```
train_data <- 2007
test_data <- 31


# Time series model

timeseries_train_load <- subset(ts_load, end = train_data)

timeseries_test_load <- subset(ts_load, start = train_data + 1, end = train_data + test_data)

# Fucntions calculations
MAPE <- function(actual, predicted) {
  mean(abs(actual - predicted) / actual) * 100
}




# Fit Naive plot
```

```
SNAIVE_fit <- snaive(timeseries_train_load, h = test_data)

autoplot(SNAIVE_fit) +
  autolayer(timeseries_test_load, series = "Actual Jul 2010",
            color = "black") +
  labs(title = "Model 1: Seasonal Naive - July 2010 Back-test") +
  theme_minimal()
```




```
accuracy(SNAIVE_fit, timeseries_test_load)
```

```
##                     ME     RMSE      MAE       MPE     MAPE     MASE      ACF1
## Training set  37.31665 949.3637 705.9425 -2.170041 19.28918 1.000000 0.7418106
## Test set     606.47849 950.3533 837.1102 12.348224 19.35913 1.185805 0.4370708
##              Theil's U
## Training set        NA
## Test set      1.772924
```

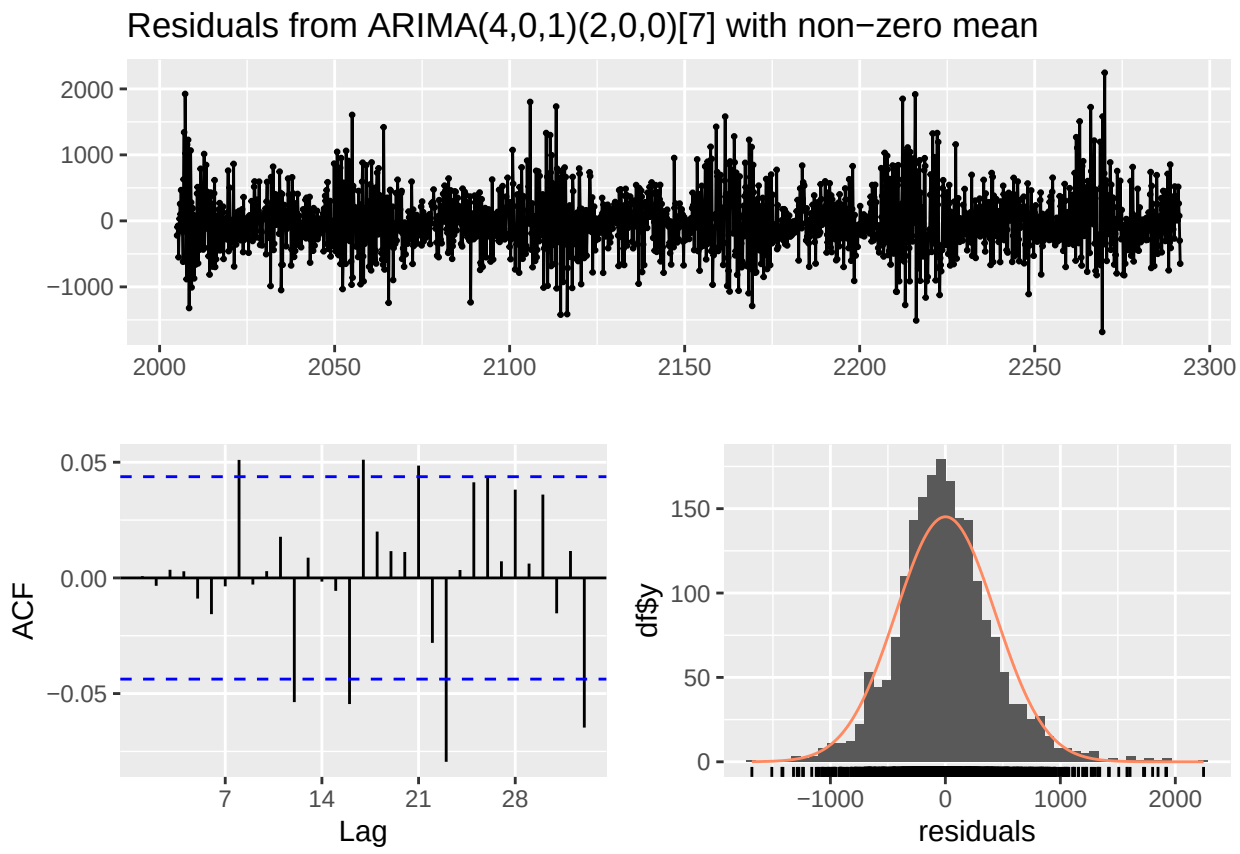
```
# MAPE
MAPE(timeseries_test_load, SNAIVE_fit$mean)
```

```
## [1] 19.35913
```

```r
ts_load_weekly <- ts(load_processed$daily_load,
                     frequency = 7,
                     start     = c(2005, 1))

# Recreate train and test using this new ts object
timeseries_train_weekly <- subset(ts_load_weekly,
                                  end   = train_data)
timeseries_test_weekly  <- subset(ts_load_weekly,
                                  start = train_data + 1,
                                  end   = train_data + test_data)


SARIMA_fitting <- auto.arima(timeseries_train_weekly,
                             stepwise = TRUE,
                             approximation = TRUE)

print(SARIMA_fitting)
```

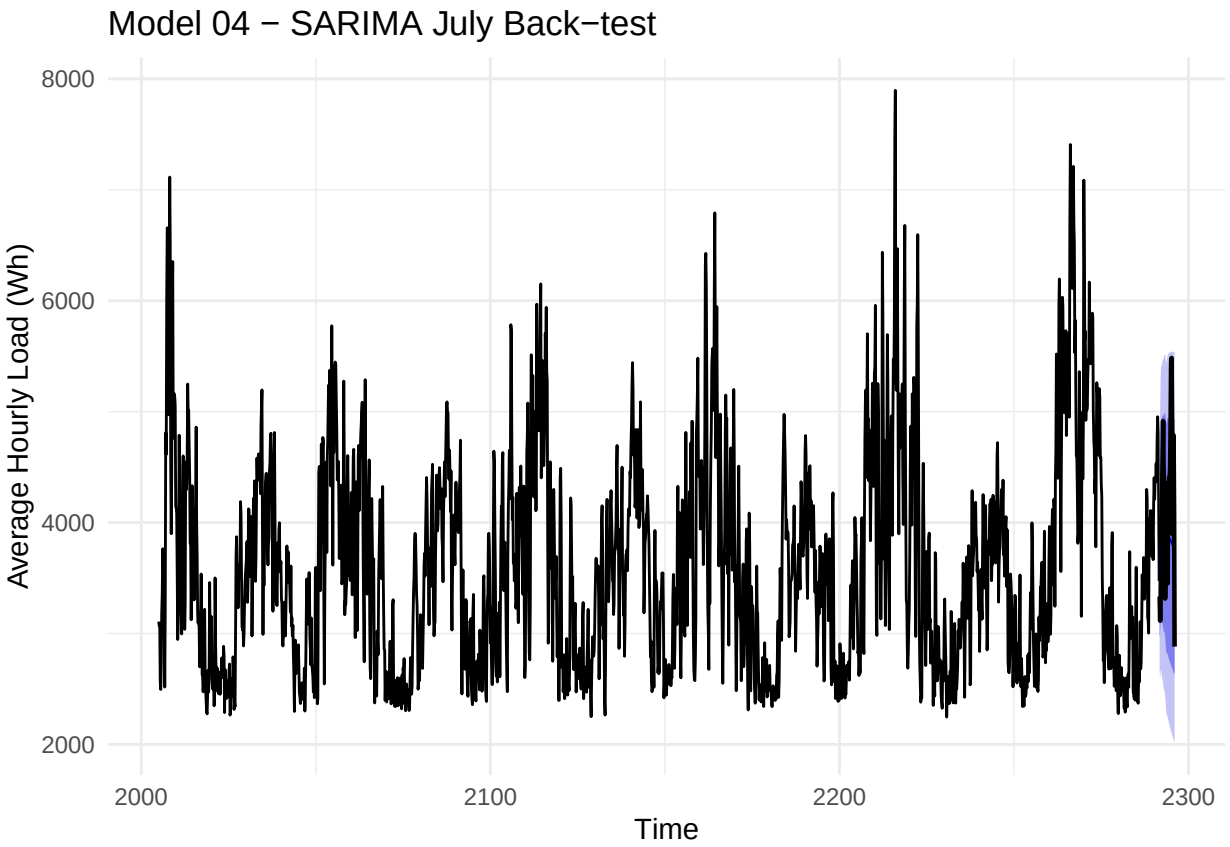
```
## Series: timeseries_train_weekly
## ARIMA(4,0,1)(2,0,0)[7] with non-zero mean
##
## Coefficients:
##          ar1      ar2     ar3      ar4      ma1     sar1    sar2       mean
##       1.7750  -1.1016  0.3370  -0.0251  -0.7845  -0.0317  0.0685  3587.6299
## s.e.  0.0509   0.0666  0.0523   0.0297   0.0457   0.0244  0.0230   144.5567
##
## sigma^2 = 188425:  log likelihood = -15033.73
## AIC=30085.46   AICc=30085.55   BIC=30135.89
```

```r
checkresiduals(SARIMA_fitting)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(4,0,1)(2,0,0)[7] with non-zero mean
## Q* = 12.652, df = 7, p-value = 0.08105
##
## Model df: 7.   Total lags used: 14
```

```r
# Forecast July 2010 back-test
SARIMA_forecasting <- forecast(object = SARIMA_fitting, h = test_data)

# Plot forecast value vs actual value

autoplot(SARIMA_forecasting) +
  autolayer(timeseries_test_weekly,
            series = "Actual July 2010",
            color = "black",
            linewidth = 0.8) +
  labs(title = "Model 04 - SARIMA July Back-test",
       x = "Time",
       y = "Average Hourly Load (Wh)") +
  theme_minimal()
```




```
# Check accuracy

accuracy(SARIMA_forecasting, timeseries_test_weekly)
```

```
##                      ME     RMSE      MAE       MPE      MAPE      MASE
## Training set   1.324977 433.2137 324.3043 -1.288172  8.988263 0.5048220
## Test set     309.628823 690.4067 576.5533  5.267758 13.515902 0.8974804
##                     ACF1 Theil's U
## Training set 0.0007294211        NA
## Test set     0.5783685413  1.257939
```

```
MAPE(as.numeric(timeseries_test_weekly),
     as.numeric(SARIMA_forecasting$mean))
```

```
## [1] 13.5159
```

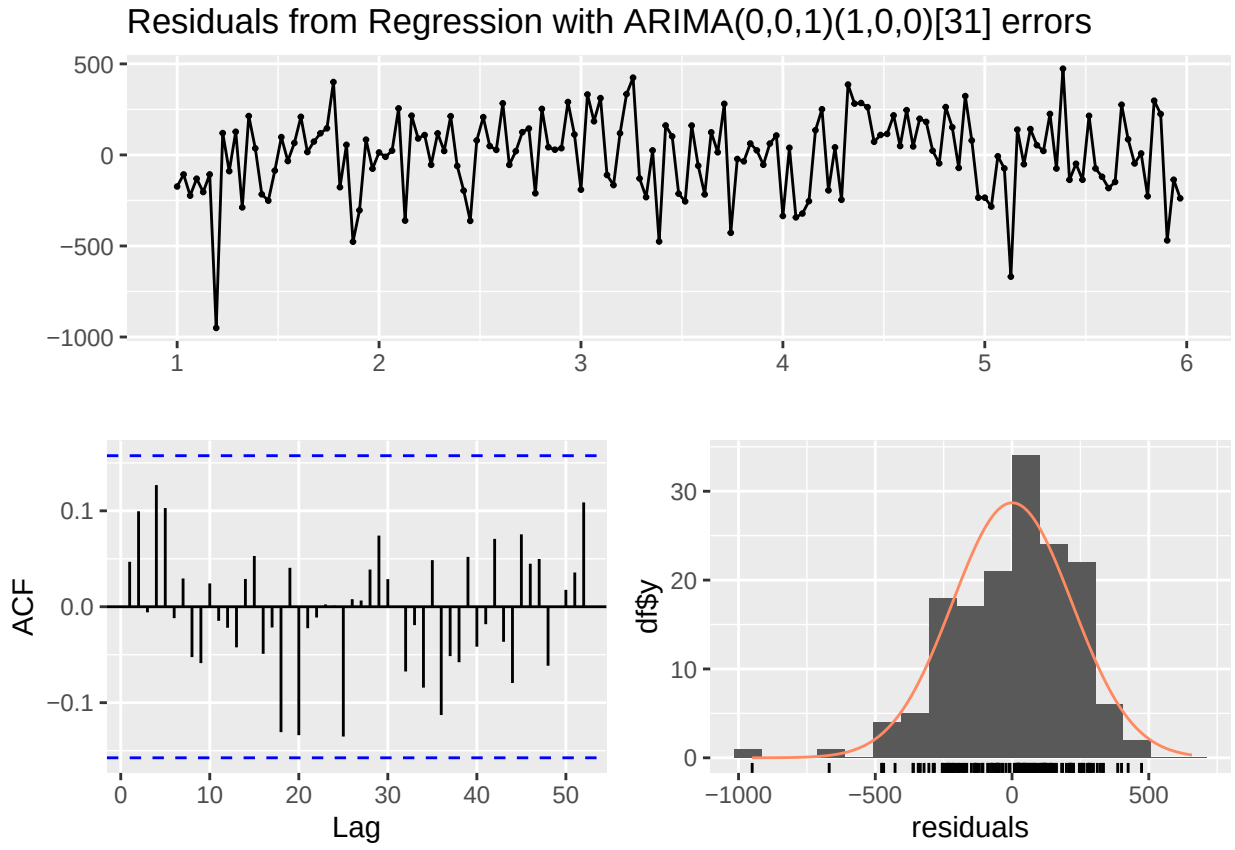
SARIMA +Temperature Model

```
july_df <- load_processed %>%
  left_join(temp_processed, by = "date") %>%
  filter(month(date) == 7) %>%
  mutate(
```

```
    temp_mean_sq = temp_mean ^ 2,
    is_weekend   = as.numeric(wday(date) %in% c(1, 7))
  )


ts_july_load <- ts(july_df$daily_load, frequency = 31)


xreg_july <- cbind(
  temp_mean    = july_df$temp_mean,
  temp_mean_sq = july_df$temp_mean_sq,
  is_weekend   = july_df$is_weekend
)


# Train = July 2005-2009 (5 years x 31 days = 155 rows)
# Test  = July 2010 (31 rows)
n_july_train <- 31 * 5
n_july_test  <- 31

ts_july_train <- subset(ts_july_load, end = n_july_train)
ts_july_test  <- subset(ts_july_load, start = n_july_train + 1)

xreg_train <- xreg_july[1:n_july_train, ]
xreg_test  <- xreg_july[(n_july_train + 1):nrow(july_df), ]

# Fit SARIMA and temp
ARIMAX_fitting <- auto.arima(
  ts_july_train,
  xreg          = xreg_train,
  seasonal      = TRUE,
  stepwise      = TRUE,
  approximation = TRUE
)

print(ARIMAX_fitting)
```

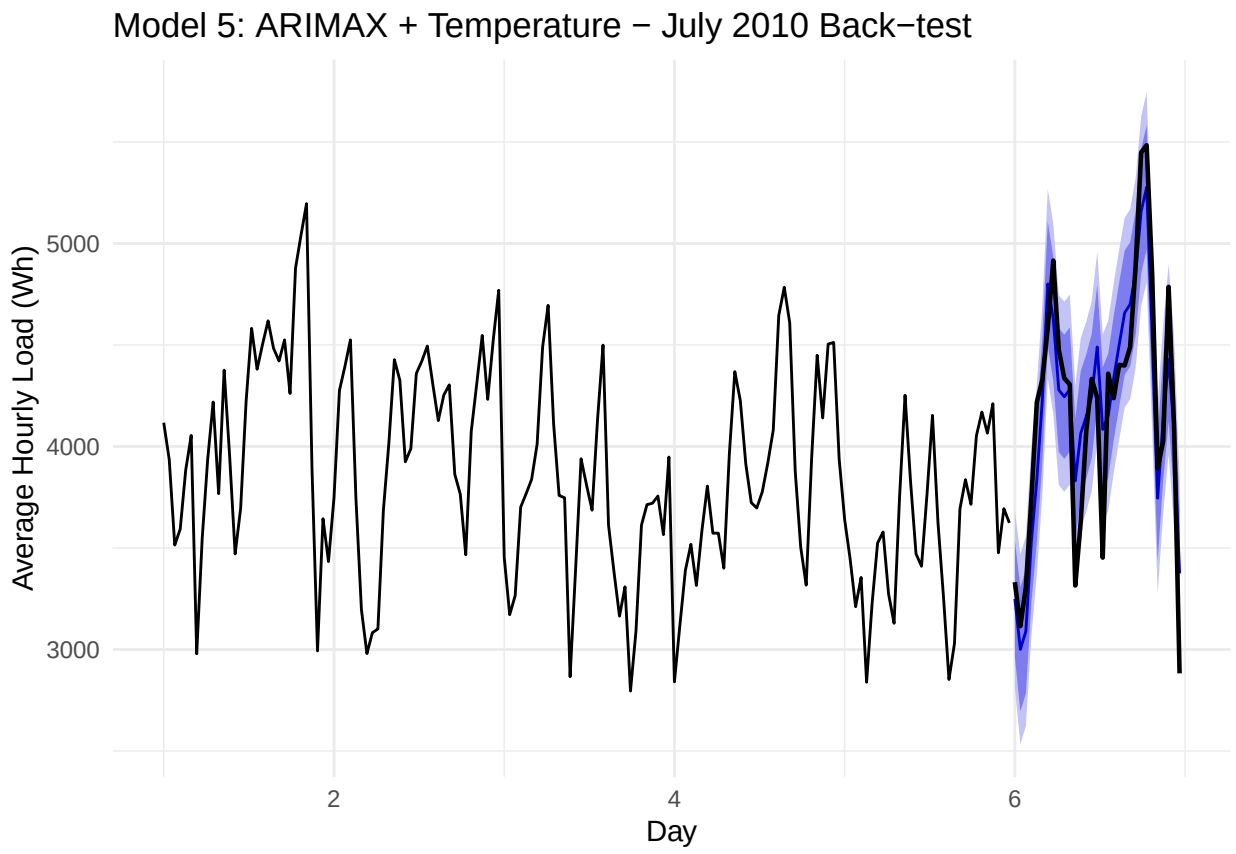
```
## Series: ts_july_train
## Regression with ARIMA(0,0,1)(1,0,0)[31] errors
##
## Coefficients:
##          ma1    sar1  temp_mean  temp_mean_sq  is_weekend
##       0.3943  0.0104   -50.8148        1.2823    145.3473
## s.e.  0.0678  0.0966     7.7242        0.0987     43.6595
##
## sigma^2 = 49296:  log likelihood = -1054.91
## AIC=2121.83   AICc=2122.4   BIC=2140.09
```

```
checkresiduals(ARIMAX_fitting)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from Regression with ARIMA(0,0,1)(1,0,0)[31] errors
## Q* = 20.724, df = 29, p-value = 0.869
##
## Model df: 2.   Total lags used: 31
```

```r
# Step 6: forecast July 2010 back-test
ARIMAX_forecasting <- forecast(
  object = ARIMAX_fitting,
  xreg   = xreg_test,
  h      = n_july_test
)

autoplot(ARIMAX_forecasting) +
  autolayer(ts_july_test,
            series    = "Actual July 2010",
            color     = "black",
            linewidth = 0.8) +
  labs(title = "Model 5: ARIMAX + Temperature - July 2010 Back-test",
       x     = "Day",
       y     = "Average Hourly Load (Wh)") +
  theme_minimal()
```




```
accuracy(ARIMAX_forecasting, ts_july_test)
```

```
##                      ME     RMSE      MAE        MPE     MAPE      MASE
## Training set  0.5176939 218.4161 170.8240 -0.3242009 4.616849 0.3021913
## Test set     -0.7617528 264.5737 217.8406 -0.5117001 5.497137 0.3853648
##                    ACF1 Theil's U
## Training set 0.04684846        NA
## Test set     0.28494268 0.5279125
```

```
MAPE(as.numeric(ts_july_test),
     as.numeric(ARIMAX_forecasting$mean))
```

```
## [1] 5.497137
```

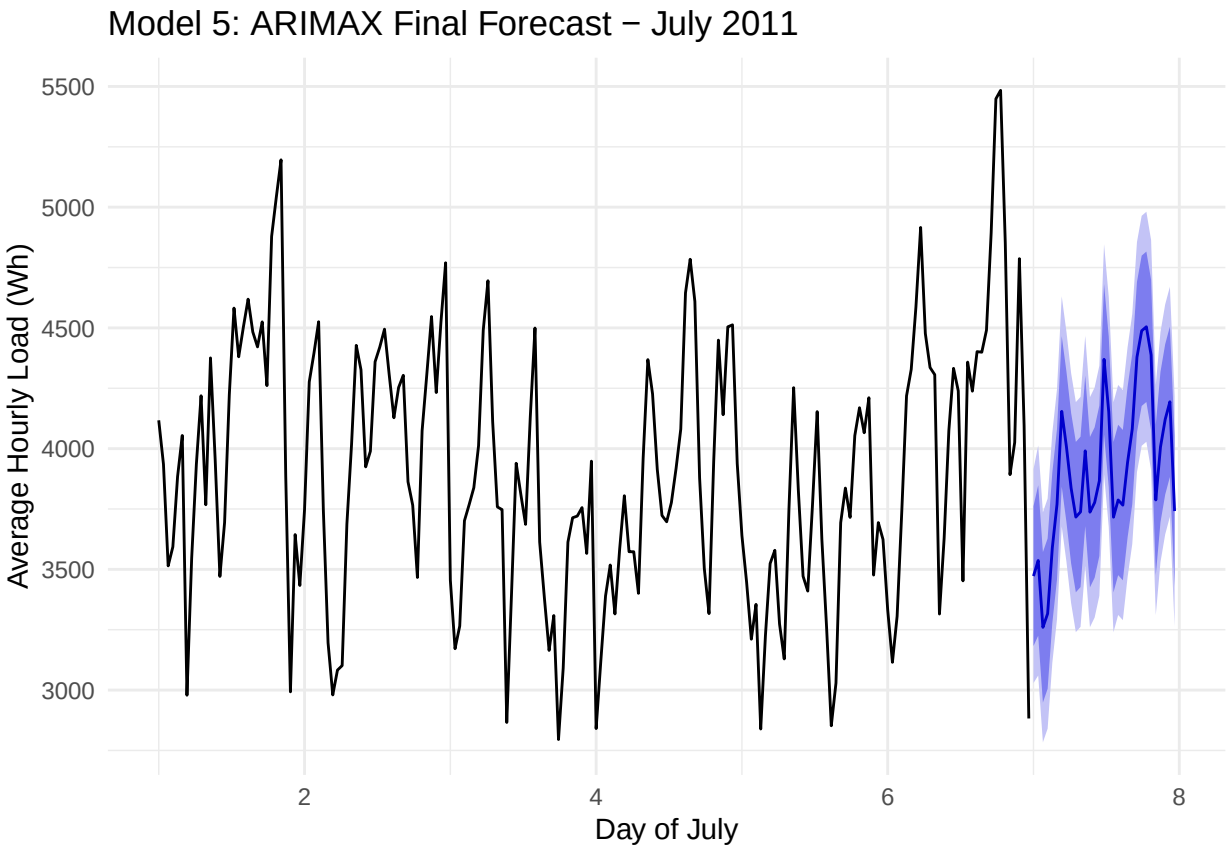
CSV. Submission

```
# July data from 2005 to 2010
ARIMAX_final <- auto.arima(
  ts_july_load,
  xreg          = xreg_july,
  seasonal      = TRUE,
  stepwise      = TRUE,
```

```
  approximation = TRUE
)

print(ARIMAX_final)
```

```
## Series: ts_july_load
## Regression with ARIMA(0,0,1)(1,0,0)[31] errors
##
## Coefficients:
##          ma1     sar1  temp_mean  temp_mean_sq  is_weekend
##       0.3949  -0.0618   -53.1261        1.3117    148.7914
## s.e.  0.0639   0.0849     6.7512        0.0858     40.3810
##
## sigma^2 = 51072:  log likelihood = -1269.74
## AIC=2551.49   AICc=2551.96   BIC=2570.84
```

```
# Use average of July 2009 and July 2010
tmean_2009 <- july_df %>%
  filter(year(date) == 2009) %>%
  pull(temp_mean)

tmean_2010 <- july_df %>%
  filter(year(date) == 2010) %>%
  pull(temp_mean)

tmean_2011 <- (tmean_2009 + tmean_2010) / 2

# Weekend flags for July
dates_july2011 <- seq.Date(as.Date("2011-07-01"),
                           as.Date("2011-07-31"),
                           by = "day")

wknd_2011 <- as.numeric(wday(dates_july2011) %in% c(1, 7))

# Forcast period
xreg_2011 <- cbind(
  temp_mean    = tmean_2011,
  temp_mean_sq = tmean_2011 ^ 2,
  is_weekend   = wknd_2011
)

# Generate forecast
ARIMAX_fc_2011 <- forecast(
  object = ARIMAX_final,
  xreg   = xreg_2011,
  h      = 31
)

autoplot(ARIMAX_fc_2011) +
  labs(title = "Model 5: ARIMAX Final Forecast - July 2011",
       x     = "Day of July",
       y     = "Average Hourly Load (Wh)") +
  theme_minimal()
```




```r
# Model submission
submission_arimax <- data.frame(
  date = format(dates_july2011, "%Y-%m-%d"),
  load = round(as.numeric(ARIMAX_fc_2011$mean), 2)
)

print(submission_arimax)
```

```
##          date    load
## 1  2011-07-01 3472.10
## 2  2011-07-02 3536.66
## 3  2011-07-03 3259.93
## 4  2011-07-04 3316.54
## 5  2011-07-05 3589.18
## 6  2011-07-06 3770.08
## 7  2011-07-07 4154.34
## 8  2011-07-08 4008.10
## 9  2011-07-09 3836.24
## 10 2011-07-10 3716.42
## 11 2011-07-11 3738.00
## 12 2011-07-12 3990.77
## 13 2011-07-13 3736.74
## 14 2011-07-14 3776.72
## 15 2011-07-15 3866.84
## 16 2011-07-16 4369.87
## 17 2011-07-17 4152.21
```

```
## 18 2011-07-18 3716.00
## 19 2011-07-19 3787.36
## 20 2011-07-20 3765.96
## 21 2011-07-21 3943.28
## 22 2011-07-22 4081.35
## 23 2011-07-23 4378.75
## 24 2011-07-24 4488.02
## 25 2011-07-25 4504.67
## 26 2011-07-26 4387.59
## 27 2011-07-27 3787.43
## 28 2011-07-28 4006.58
## 29 2011-07-29 4121.19
## 30 2011-07-30 4194.04
## 31 2011-07-31 3741.40
```

```
write.csv(submission_arimax,
          "./output/submission_arimatemp.csv",
          row.names = FALSE)

cat("Upload output/submission_arimax.csv to Kaggle.\n")
```

```
## Upload output/submission_arimax.csv to Kaggle.
```

```
# New submission
# Force remove old forecast object first
rm(ARIMAX_fc_2011)

# Rebuild xreg_2011 fresh
dates_july2011 <- seq.Date(as.Date("2011-07-01"),
                           as.Date("2011-07-31"),
                           by = "day")

tmean_2009 <- july_df %>% filter(year(date) == 2009) %>% pull(temp_mean)
tmean_2010 <- july_df %>% filter(year(date) == 2010) %>% pull(temp_mean)
tmean_2011 <- (tmean_2009 + tmean_2010) / 2
wknd_2011  <- as.numeric(wday(dates_july2011) %in% c(1, 7))

xreg_2011 <- cbind(
  temp_mean    = tmean_2011,
  temp_mean_sq = tmean_2011 ^ 2,
  is_weekend   = wknd_2011
)

# Print xreg_2011 to verify temperatures look right
cat("xreg_2011 temperatures:\n")
```

```
## xreg_2011 temperatures:
```

```
print(round(xreg_2011, 2))
```

```
##       temp_mean temp_mean_sq is_weekend
##  [1,]     76.75      5889.88          0
```

```
##  [2,]     75.01      5626.79          1
##  [3,]     73.10      5343.02          1
##  [4,]     74.55      5558.12          0
##  [5,]     76.50      5852.36          0
##  [6,]     77.60      6021.60          0
##  [7,]     79.97      6395.36          0
##  [8,]     79.23      6277.85          0
##  [9,]     77.09      5942.53          1
## [10,]     76.23      5811.45          1
## [11,]     77.35      5983.78          0
## [12,]     78.80      6209.89          0
## [13,]     77.16      5953.43          0
## [14,]     77.57      6017.56          0
## [15,]     78.23      6120.15          0
## [16,]     80.38      6461.34          1
## [17,]     78.84      6215.28          1
## [18,]     77.28      5972.62          0
## [19,]     77.63      6026.56          0
## [20,]     77.49      6004.17          0
## [21,]     78.59      6176.39          0
## [22,]     79.51      6321.31          0
## [23,]     80.55      6488.52          1
## [24,]     81.33      6614.87          1
## [25,]     82.32      6777.19          0
## [26,]     81.64      6665.19          0
## [27,]     77.74      6043.10          0
## [28,]     79.07      6251.82          0
## [29,]     79.99      6397.74          0
## [30,]     79.37      6299.33          1
## [31,]     76.16      5800.57          1
```

```r
# Now generate forecast fresh
ARIMAX_fc_2011 <- forecast(
  object = ARIMAX_final,
  xreg   = xreg_2011,
  h      = 31
)

# Check values
cat("\nForecast values:\n")
```

```
##
## Forecast values:
```

```r
print(round(as.numeric(ARIMAX_fc_2011$mean), 2))
```

```
##  [1] 3472.10 3536.66 3259.93 3316.54 3589.18 3770.08 4154.34 4008.10 3836.24
## [10] 3716.42 3738.00 3990.77 3736.74 3776.72 3866.84 4369.87 4152.21 3716.00
## [19] 3787.36 3765.96 3943.28 4081.35 4378.75 4488.02 4504.67 4387.59 3787.43
## [28] 4006.58 4121.19 4194.04 3741.40
```

```r
# Model diagnostic

# Check if ARIMAX_final and SARIMA_fitting produce same fitted values
cat("ARIMAX_final fitted (last 5):\n")
```

```
## ARIMAX_final fitted (last 5):
```

```r
print(round(tail(as.numeric(ARIMAX_final$fitted), 5), 2))
```

```
## [1] 3821.79 4140.47 4419.06 4243.28 3330.13
```

```r
cat("\nSARIMA_fitting fitted (last 5):\n")
```

```
##
## SARIMA_fitting fitted (last 5):
```

```r
print(round(tail(as.numeric(SARIMA_fitting$fitted), 5), 2))
```

```
## [1] 4247.47 4690.72 4741.68 4579.21 4123.11
```

```r
# Check column names match EXACTLY between training and forecast xreg
cat("\nTraining xreg column names:\n")
```

```
##
## Training xreg column names:
```

```r
print(colnames(xreg_july))
```

```
## [1] "temp_mean"    "temp_mean_sq" "is_weekend"
```

```r
cat("\nForecast xreg column names:\n")
```

```
##
## Forecast xreg column names:
```

```r
print(colnames(xreg_2011))
```

```
## [1] "temp_mean"    "temp_mean_sq" "is_weekend"
```

```r
# Check if ARIMAX_final$coef contains temperature coefficients
cat("\nARIMAX_final coefficients:\n")
```

```
##
## ARIMAX_final coefficients:
```

```r
print(ARIMAX_final$coef)
```

```
##          ma1         sar1    temp_mean temp_mean_sq   is_weekend
##   0.39492842  -0.06182939 -53.12614884   1.31169688 148.79137183
```

```r
# Most importantly - check what ts_july_load looks like
cat("\nts_july_load length:", length(ts_july_load), "\n")
```

```
##
## ts_july_load length: 186
```

```r
cat("ts_july_load last 5 values:\n")
```

```
## ts_july_load last 5 values:
```

```r
print(round(tail(as.numeric(ts_july_load), 5), 2))
```

```
## [1] 3892.33 4025.62 4786.62 4094.83 2882.38
```

```r
# Alternative approach using predict() directly
ARIMAX_fc_2011 <- predict(ARIMAX_final,
                          n.ahead = 31,
                          newxreg = xreg_2011)

# predict() returns $pred (point forecast) not $mean
cat("Forecast values using predict():\n")
```

```
## Forecast values using predict():
```

```r
print(round(as.numeric(ARIMAX_fc_2011$pred), 2))
```

```
##  [1] 3472.10 3536.66 3259.93 3316.54 3589.18 3770.08 4154.34 4008.10 3836.24
## [10] 3716.42 3738.00 3990.77 3736.74 3776.72 3866.84 4369.87 4152.21 3716.00
## [19] 3787.36 3765.96 3943.28 4081.35 4378.75 4488.02 4504.67 4387.59 3787.43
## [28] 4006.58 4121.19 4194.04 3741.40
```

```r
# Test with completely fake temperatures to prove xreg is working
xreg_test_fake <- cbind(
  temp_mean    = rep(100, 31),  # extreme fake temp
  temp_mean_sq = rep(10000, 31),
  is_weekend   = rep(0, 31)
)

fc_fake <- forecast(ARIMAX_final, xreg = xreg_test_fake, h = 31)
cat("Forecast with fake temp=100:\n")
```

```
## Forecast with fake temp=100:
```

```r
print(round(as.numeric(fc_fake$mean), 2))
```

```
##  [1] 7627.92 7796.68 7790.37 7791.02 7781.20 7798.45 7818.47 7787.16 7792.38
## [10] 7799.07 7803.02 7836.13 7831.13 7809.01 7799.55 7820.53 7843.52 7791.81
## [19] 7810.92 7811.22 7821.26 7817.94 7802.72 7787.73 7792.95 7786.49 7794.96
## [28] 7811.04 7782.99 7803.33 7834.54
```

```r
# MANUAL FORECAST


# Extract coefficients from the fitted model
coef_ma1    <- ARIMAX_final$coef["ma1"]
coef_temp   <- ARIMAX_final$coef["temp_mean"]
coef_tempsq <- ARIMAX_final$coef["temp_mean_sq"]
coef_wknd   <- ARIMAX_final$coef["is_weekend"]

cat("Model coefficients:\n")
```

```
## Model coefficients:
```

```r
cat("  temp_mean:   ", round(coef_temp, 4), "\n")
```

```
##   temp_mean:    -53.1261
```

```r
cat("  temp_mean_sq:", round(coef_tempsq, 4), "\n")
```

```
##   temp_mean_sq: 1.3117
```

```r
cat("  is_weekend:  ", round(coef_wknd, 4), "\n")
```

```
##   is_weekend:   148.7914
```

```r
# Compute the regression part of the forecast
# load = intercept + b1*temp + b2*temp^2 + b3*weekend
regression_fc <- coef_temp   * xreg_2011[,"temp_mean"] +
                 coef_tempsq * xreg_2011[,"temp_mean_sq"] +
                 coef_wknd   * xreg_2011[,"is_weekend"]

# Add the mean (intercept equivalent from ARIMA with mean)
# Get it from the last fitted values trend
mean_load <- mean(as.numeric(ts_july_load))
cat("\nHistorical July mean load:", round(mean_load, 2), "\n")
```

```
##
## Historical July mean load: 3905.84
```

```r
manual_forecast <- mean_load + regression_fc -
                   mean(coef_temp   * july_df$temp_mean +
                        coef_tempsq * july_df$temp_mean_sq +
                        coef_wknd   * july_df$is_weekend)

cat("\nManual forecast values:\n")
```

```
##
## Manual forecast values:
```

```
print(round(manual_forecast, 2))
```

```
##  [1] 3648.77 3544.56 3274.14 3330.10 3612.56 3776.20 4140.45 4025.53 3848.44
## [10] 3721.92 3739.57 3959.22 3710.19 3772.29 3871.88 4353.92 4113.27 3728.76
## [19] 3781.02 3759.32 3926.60 4067.99 4380.60 4504.87 4516.30 4405.69 3797.05
## [28] 4000.12 4142.78 4195.29 3711.44
```

```
cat("\nManual forecast mean:", round(mean(manual_forecast), 2), "\n")
```

```
##
## Manual forecast mean: 3914.87
```

```
cat("Historical July mean:", round(mean_load, 2), "\n")
```

```
## Historical July mean: 3905.84
```

```
# Save this as your submission
submission_arimax_v2 <- data.frame(
  date = format(dates_july2011, "%Y-%m-%d"),
  load = round(manual_forecast, 2)
)

print(submission_arimax_v2)
```

```
##          date    load
## 1  2011-07-01 3648.77
## 2  2011-07-02 3544.56
## 3  2011-07-03 3274.14
## 4  2011-07-04 3330.10
## 5  2011-07-05 3612.56
## 6  2011-07-06 3776.20
## 7  2011-07-07 4140.45
## 8  2011-07-08 4025.53
## 9  2011-07-09 3848.44
## 10 2011-07-10 3721.92
## 11 2011-07-11 3739.57
## 12 2011-07-12 3959.22
## 13 2011-07-13 3710.19
## 14 2011-07-14 3772.29
## 15 2011-07-15 3871.88
## 16 2011-07-16 4353.92
## 17 2011-07-17 4113.27
## 18 2011-07-18 3728.76
## 19 2011-07-19 3781.02
## 20 2011-07-20 3759.32
## 21 2011-07-21 3926.60
## 22 2011-07-22 4067.99
## 23 2011-07-23 4380.60
```

```
## 24 2011-07-24 4504.87
## 25 2011-07-25 4516.30
## 26 2011-07-26 4405.69
## 27 2011-07-27 3797.05
## 28 2011-07-28 4000.12
## 29 2011-07-29 4142.78
## 30 2011-07-30 4195.29
## 31 2011-07-31 3711.44
```

```
write.csv(submission_arimax_v2,
          "./output/submission_sarimatemp_02.csv",
          row.names = FALSE)
```

NEW SUBMISSION

Setup

```
library(readxl)
library(lubridate)
library(ggplot2)
library(forecast)
library(tseries)
library(tidyverse)
library(cowplot)
```

Load and Process Datasets

```
# Load raw Excel files
raw_load_data <- read_excel(
  "~/Documents/ENVIRON 797 - TSA Spring 2026/Forecasting2026/data/load.xlsx")
raw_temperature_data <- read_excel(
  "~/Documents/ENVIRON 797 - TSA Spring 2026/Forecasting2026/data/temperature.xlsx")
raw_relativehumidity_data <- read_excel(
  "~/Documents/ENVIRON 797 - TSA Spring 2026/Forecasting2026/data/relative_humidity.xlsx")
submission_template <- read.csv(
  "~/Documents/ENVIRON 797 - TSA Spring 2026/Forecasting2026/data/submission_template.csv")

# Process load - daily TOTAL (rowSums not rowMeans to match Kaggle format)
load_processed <- raw_load_data %>%
  mutate(
    date       = as.Date(date),
    daily_load = rowSums(across(starts_with("h")), na.rm = TRUE)
  ) %>%
  select(date, daily_load) %>%
  arrange(date)

# Process temperature - average across 28 stations then daily average
temp_cols <- grep("^t_ws", names(raw_temperature_data), value = TRUE)
```

```r
temp_processed <- raw_temperature_data %>%
  mutate(
    date     = as.Date(date),
    avg_temp = rowMeans(across(all_of(temp_cols)), na.rm = TRUE)
  ) %>%
  filter(!is.na(date)) %>%
  group_by(date) %>%
  summarise(
    temp_mean = mean(avg_temp, na.rm = TRUE),
    temp_max  = max(avg_temp,  na.rm = TRUE),
    .groups   = "drop"
  )

# Process humidity
rh_cols <- grep("^rh_ws", names(raw_relativehumidity_data), value = TRUE)

humid_processed <- raw_relativehumidity_data %>%
  mutate(
    date   = as.Date(date),
    avg_rh = rowMeans(across(all_of(rh_cols)), na.rm = TRUE)
  ) %>%
  group_by(date) %>%
  summarise(
    rh_mean = mean(avg_rh, na.rm = TRUE),
    .groups = "drop"
  )

head(load_processed)
```

```
## # A tibble: 6 x 2
##   date       daily_load
##   <date>          <dbl>
## 1 2005-01-01      74583
## 2 2005-01-02      73639
## 3 2005-01-03      73471
## 4 2005-01-04      61577
## 5 2005-01-05      59897
## 6 2005-01-06      65638
```

```r
head(temp_processed)
```

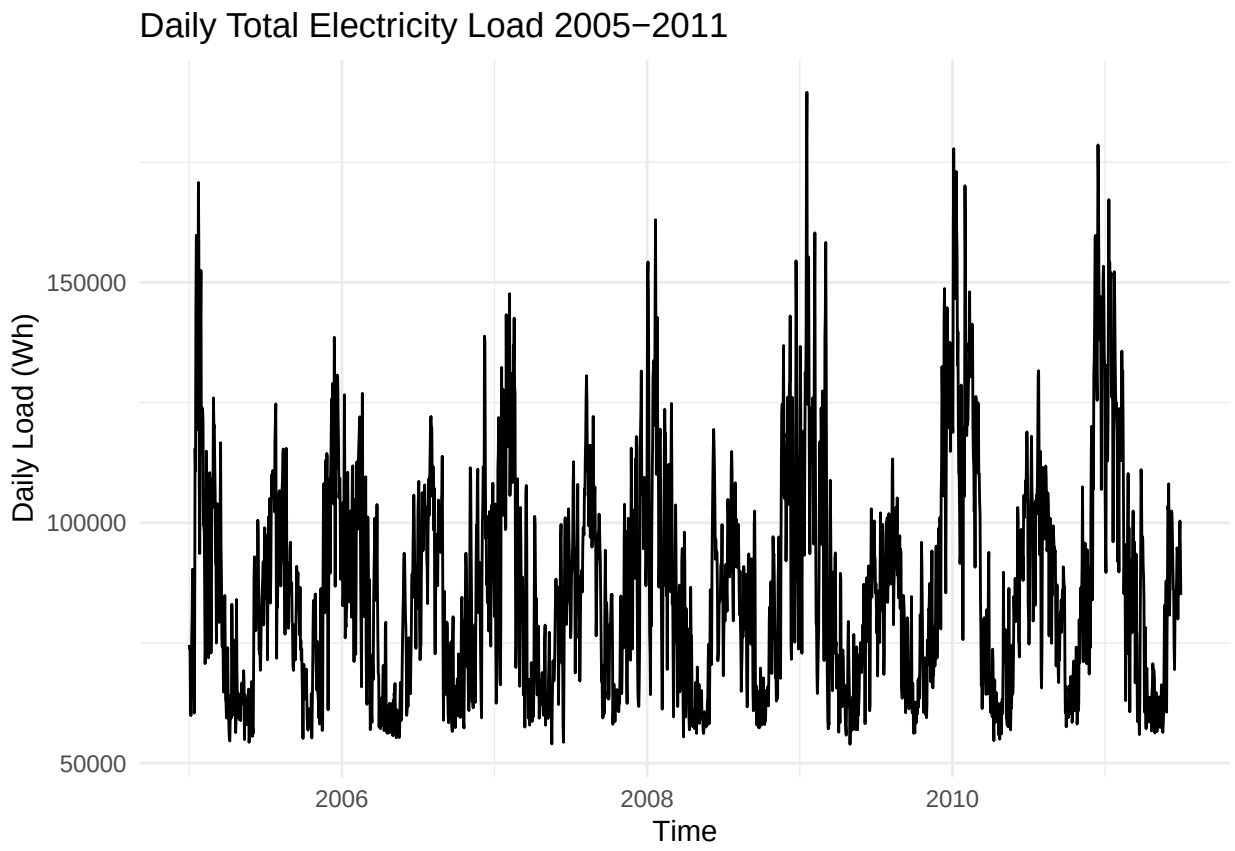
```
## # A tibble: 6 x 3
##   date       temp_mean temp_max
##   <date>         <dbl>    <dbl>
## 1 2005-01-01      53.6     68.2
## 2 2005-01-02      53.8     65.7
## 3 2005-01-03      55.9     67.3
## 4 2005-01-04      61.7     72
## 5 2005-01-05      60.4     68.6
## 6 2005-01-06      62.0     68.1
```

```r
summary(load_processed)
```

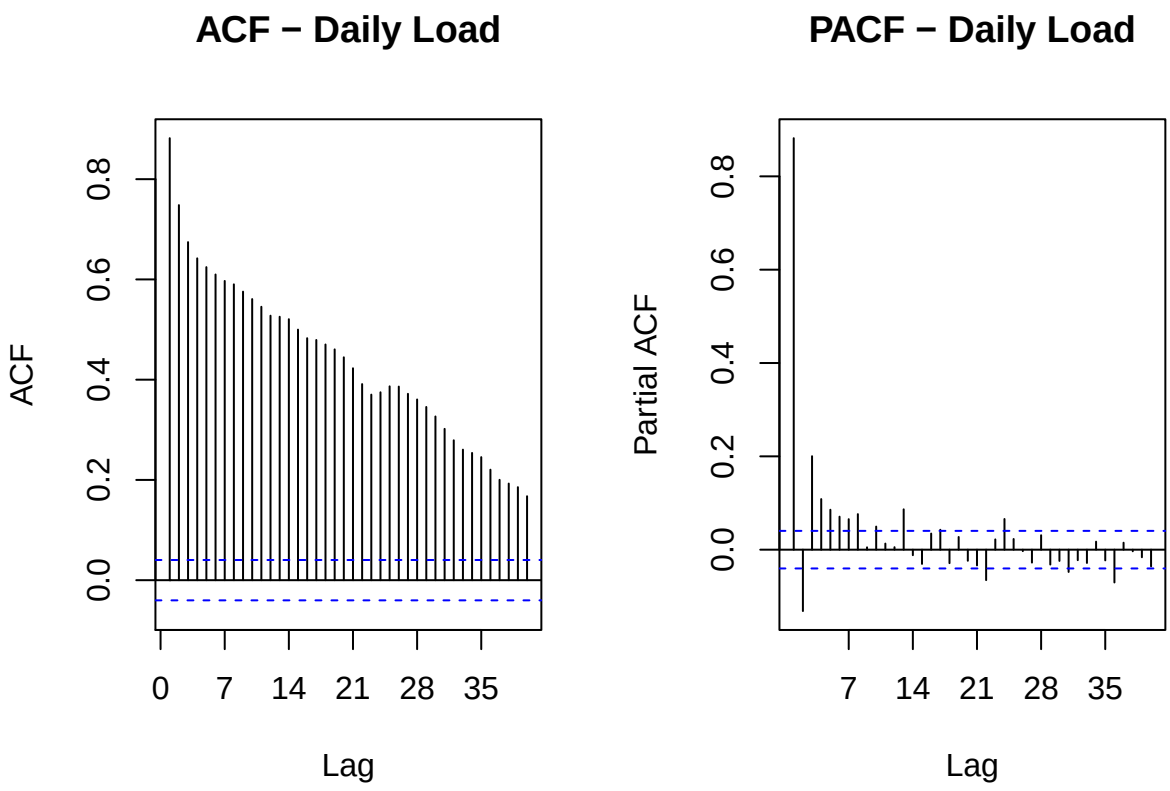
```
##       date              daily_load
##  Min.   :2005-01-01   Min.   : 53927
##  1st Qu.:2006-08-16   1st Qu.: 67158
##  Median :2008-03-31   Median : 84154
##  Mean   :2008-03-31   Mean   : 87083
##  3rd Qu.:2009-11-14   3rd Qu.:101059
##  Max.   :2011-06-30   Max.   :189518
```

Initial Plots

```r
# Create msts object - two seasonal periods: weekly (7) and annual (365)
ts_load <- msts(
  load_processed$daily_load,
  seasonal.periods = c(7, 365),
  start = c(2005, 1)
)

# Time series plot
autoplot(ts_load) +
  labs(title = "Daily Total Electricity Load 2005-2011",
       x = "Time", y = "Daily Load (Wh)") +
  theme_minimal()
```



```r
# ACF and PACF
par(mfrow = c(1, 2))
Acf(ts_load,  lag = 40, plot = TRUE, main = "ACF - Daily Load")
Pacf(ts_load, lag = 40,              main = "PACF - Daily Load")
```




```r
par(mfrow = c(1, 1))
```

Train/Test Split

```r
# Train = Jan 2005 to Jun 2010 (2007 rows)
# Test  = Jul 2010 (31 rows) - back-test validation period
train_data <- sum(load_processed$date <= as.Date("2010-06-30"))
test_data  <- sum(load_processed$date >= as.Date("2010-07-01") &
                  load_processed$date <= as.Date("2010-07-31"))

cat("Training rows:", train_data, "\n")
```

```
## Training rows: 2007
```

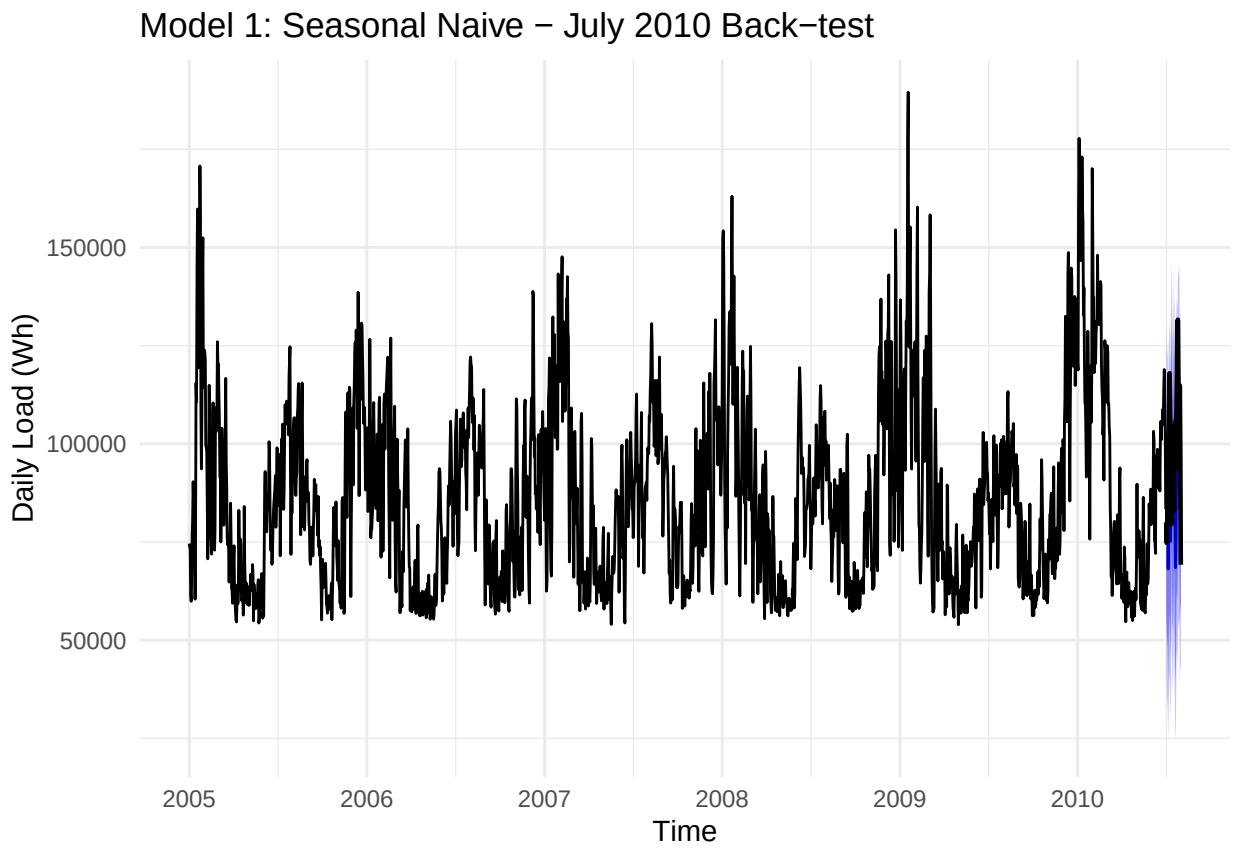
```r
cat("Test rows:    ", test_data,  "\n")
```

```
## Test rows:     31
```

```r
# MAPE function - same metric Kaggle uses
MAPE <- function(actual, predicted) {
  mean(abs(actual - predicted) / actual) * 100
}
```

Model 1 — Seasonal Naive

```r
timeseries_train_load <- subset(ts_load, end = train_data)
timeseries_test_load  <- subset(ts_load,
                                start = train_data + 1,
                                end   = train_data + test_data)

SNAIVE_fit <- snaive(timeseries_train_load, h = test_data)

autoplot(SNAIVE_fit) +
  autolayer(timeseries_test_load, series = "Actual Jul 2010",
            color = "black", linewidth = 0.8) +
  labs(title = "Model 1: Seasonal Naive - July 2010 Back-test",
       x = "Time", y = "Daily Load (Wh)") +
  theme_minimal()
```




```r
accuracy(SNAIVE_fit, timeseries_test_load)
```

```
##                      ME     RMSE      MAE       MPE     MAPE     MASE      ACF1
## Training set   895.6352 22782.74 16941.56 -2.170594 19.29074 1.000000 0.7419935
## Test set     14555.4839 22808.48 20090.65 12.348224 19.35913 1.185879 0.4370708
##              Theil's U
## Training set        NA
## Test set      1.772924
```

```
MAPE(as.numeric(timeseries_test_load),
     as.numeric(SNAIVE_fit$mean))
```

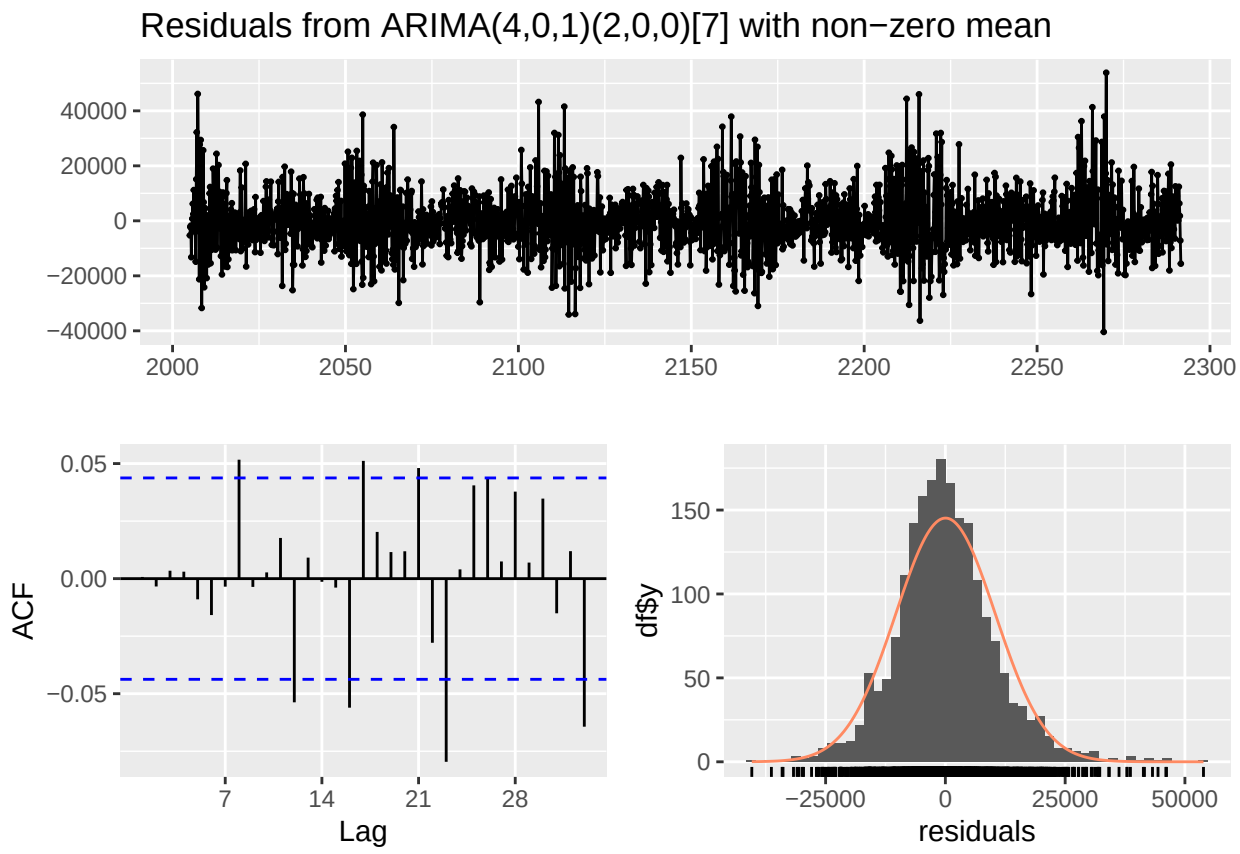
```
## [1] 19.35913
```

Model 2 — SARIMA

```
# Use weekly ts object - auto.arima cannot handle msts directly
ts_load_weekly <- ts(load_processed$daily_load,
                     frequency = 7, start = c(2005, 1))

timeseries_train_weekly <- subset(ts_load_weekly, end = train_data)
timeseries_test_weekly  <- subset(ts_load_weekly,
                                  start = train_data + 1,
                                  end   = train_data + test_data)

SARIMA_fitting <- auto.arima(timeseries_train_weekly,
                             stepwise = TRUE, approximation = TRUE)
print(SARIMA_fitting)
```

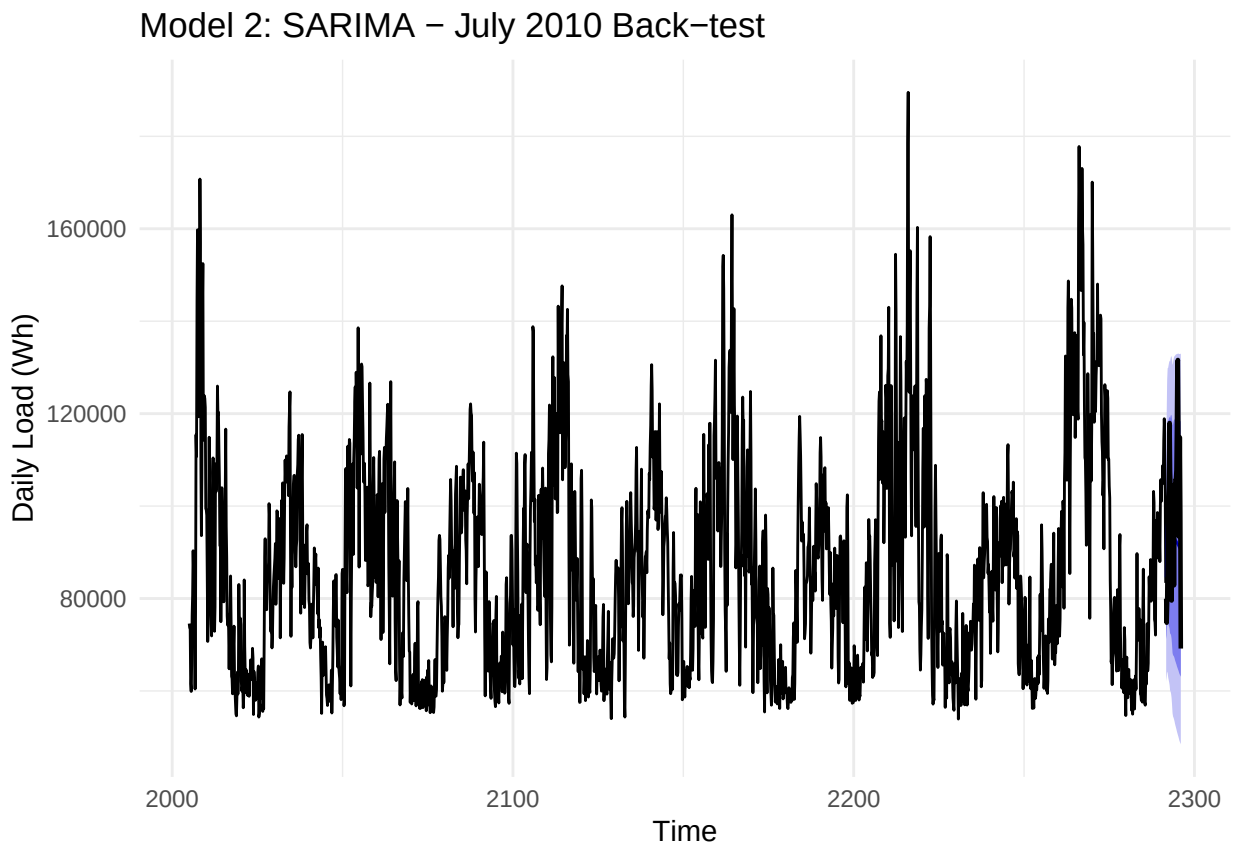
```
## Series: timeseries_train_weekly
## ARIMA(4,0,1)(2,0,0)[7] with non-zero mean
##
## Coefficients:
##          ar1      ar2     ar3      ar4      ma1     sar1    sar2       mean
##       1.7743  -1.1007  0.3363  -0.0246  -0.7833  -0.0338  0.0663  86093.839
## s.e.  0.0510   0.0667  0.0523   0.0297   0.0458   0.0244  0.0230   3468.806
##
## sigma^2 = 108463601:  log likelihood = -21411.44
## AIC=42840.88   AICc=42840.97   BIC=42891.32
```

```
checkresiduals(SARIMA_fitting)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(4,0,1)(2,0,0)[7] with non-zero mean
## Q* = 12.826, df = 7, p-value = 0.07645
##
## Model df: 7.   Total lags used: 14
```

```
SARIMA_forecasting <- forecast(object = SARIMA_fitting, h = test_data)

autoplot(SARIMA_forecasting) +
  autolayer(timeseries_test_weekly, series = "Actual July 2010",
            color = "black", linewidth = 0.8) +
  labs(title = "Model 2: SARIMA - July 2010 Back-test",
       x = "Time", y = "Daily Load (Wh)") +
  theme_minimal()
```




```
accuracy(SARIMA_forecasting, timeseries_test_weekly)
```

```
##                      ME     RMSE       MAE       MPE      MAPE      MASE
## Training set   31.76819 10393.81  7779.632 -1.287969  8.986091 0.5044691
## Test set     7450.37484 16573.67 13841.564  5.287863 13.517862 0.8975543
##                      ACF1 Theil's U
## Training set 0.0007221839        NA
## Test set     0.5781258625  1.258022
```

```
MAPE(as.numeric(timeseries_test_weekly),
     as.numeric(SARIMA_forecasting$mean))
```

```
## [1] 13.51786
```

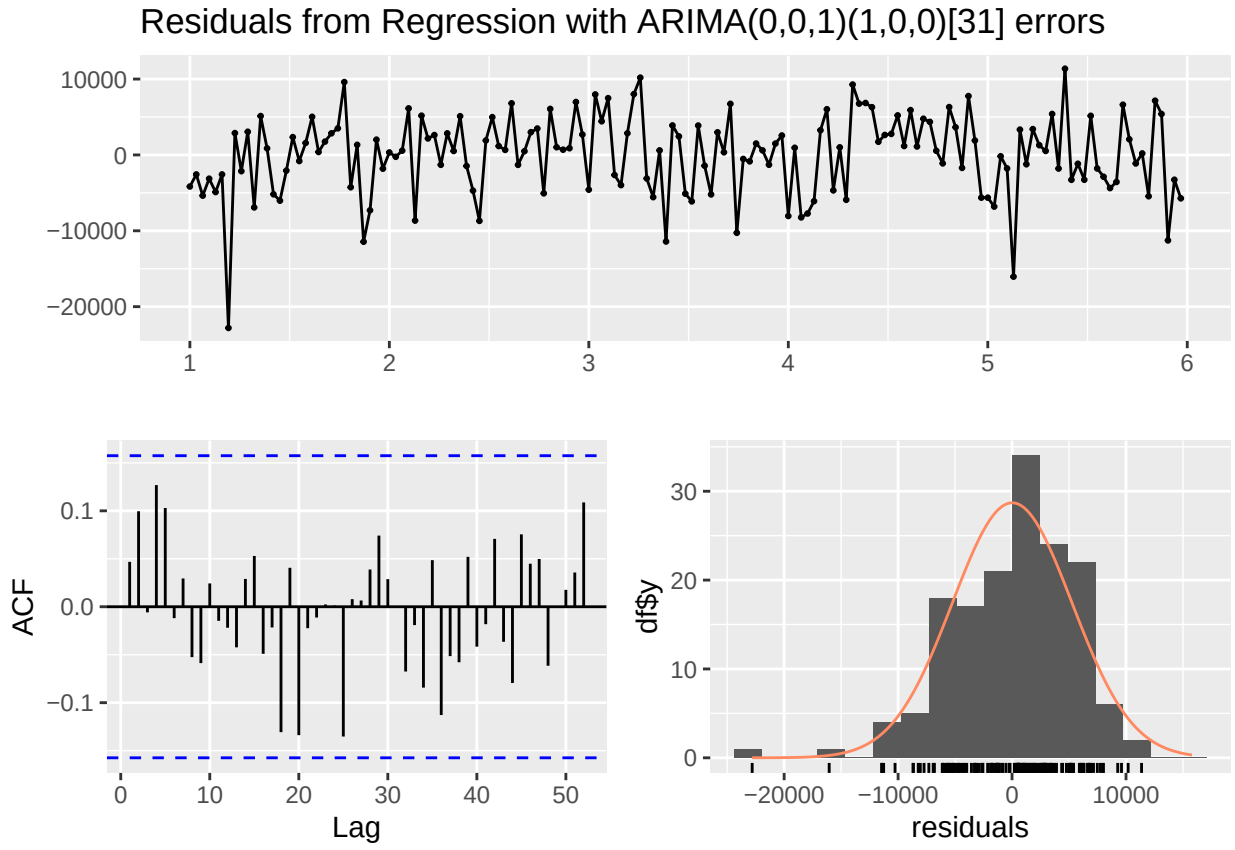
Model 3 — SARIMA + Temperature (ARIMAX)

```
# Merge load and temperature, filter to July only
july_df <- load_processed %>%
  left_join(temp_processed, by = "date") %>%
  filter(month(date) == 7) %>%
  mutate(
    temp_mean_sq = temp_mean ^ 2,
```

```r
    is_weekend   = as.numeric(wday(date) %in% c(1, 7))
  )

# ts object for July load
ts_july_load <- ts(july_df$daily_load, frequency = 31)

# External regressor matrix
xreg_july <- cbind(
  temp_mean    = july_df$temp_mean,
  temp_mean_sq = july_df$temp_mean_sq,
  is_weekend   = july_df$is_weekend
)

# Train = July 2005-2009, Test = July 2010
n_july_train <- 31 * 5
n_july_test  <- 31

ts_july_train <- subset(ts_july_load, end = n_july_train)
ts_july_test  <- subset(ts_july_load, start = n_july_train + 1)
xreg_train    <- xreg_july[1:n_july_train, ]
xreg_test     <- xreg_july[(n_july_train + 1):nrow(july_df), ]

# Fit ARIMAX
ARIMAX_fitting <- auto.arima(ts_july_train, xreg = xreg_train,
                             seasonal = TRUE, stepwise = TRUE,
                             approximation = TRUE)
print(ARIMAX_fitting)
```

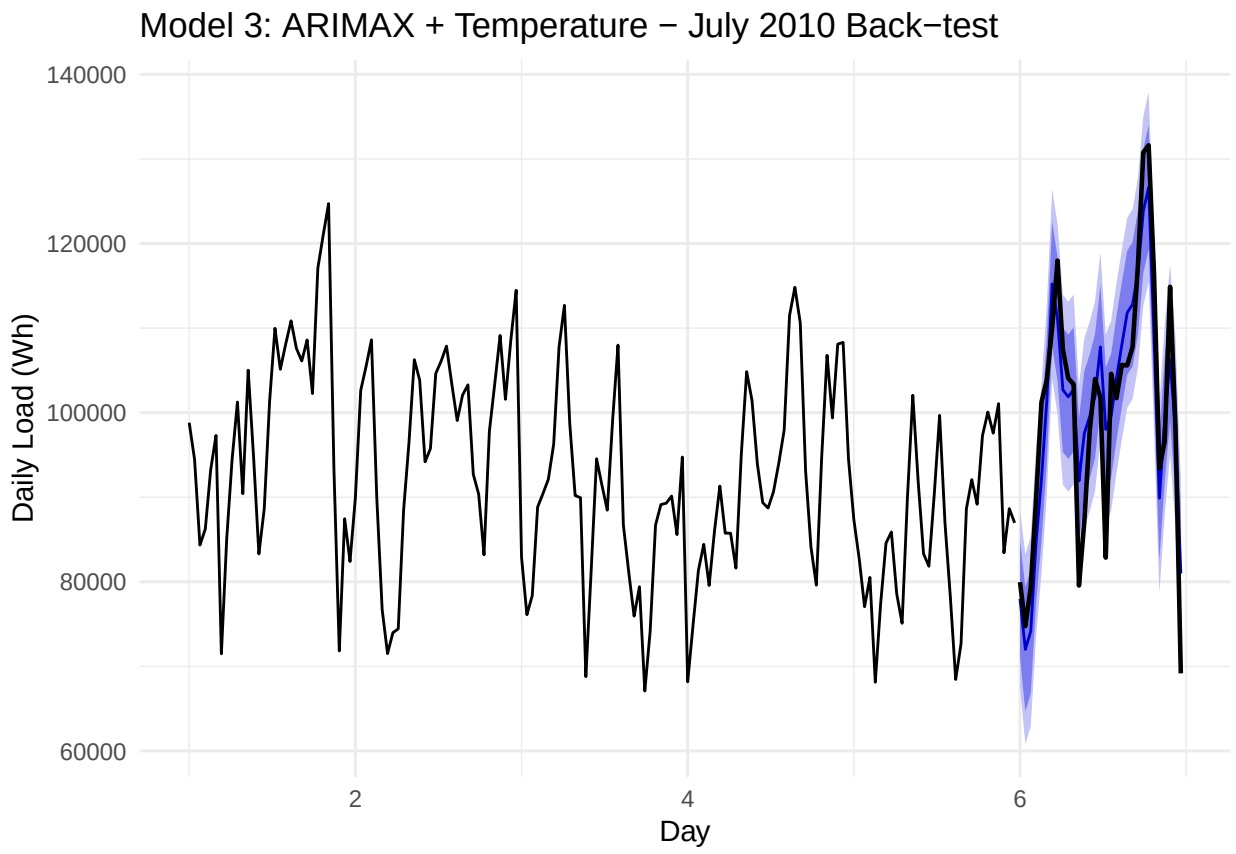
```
## Series: ts_july_train
## Regression with ARIMA(0,0,1)(1,0,0)[31] errors
##
## Coefficients:
##          ma1    sar1   temp_mean  temp_mean_sq  is_weekend
##       0.3943  0.0104  -1219.7622       30.7772    3488.900
## s.e.  0.0678  0.0966    185.3856        2.3691    1047.838
##
## sigma^2 = 28394351:  log likelihood = -1547.51
## AIC=3107.02   AICc=3107.59   BIC=3125.29
```

```r
checkresiduals(ARIMAX_fitting)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from Regression with ARIMA(0,0,1)(1,0,0)[31] errors
## Q* = 20.724, df = 29, p-value = 0.869
##
## Model df: 2.   Total lags used: 31
```

```
# Back-test forecast
ARIMAX_forecasting <- forecast(object = ARIMAX_fitting,
                               xreg = xreg_test, h = n_july_test)

autoplot(ARIMAX_forecasting) +
  autolayer(ts_july_test, series = "Actual July 2010",
            color = "black", linewidth = 0.8) +
  labs(title = "Model 3: ARIMAX + Temperature - July 2010 Back-test",
       x = "Day", y = "Daily Load (Wh)") +
  theme_minimal()
```




```
accuracy(ARIMAX_forecasting, ts_july_test)
```

```
##                     ME     RMSE      MAE        MPE     MAPE      MASE
## Training set  12.42205 5241.985 4099.791 -0.3241351 4.616876 0.3021925
## Test set     -18.66464 6349.831 5228.212 -0.5119758 5.497257 0.3853674
##                    ACF1 Theil's U
## Training set 0.04679047        NA
## Test set     0.28495488 0.5279266
```

```
MAPE(as.numeric(ts_july_test),
     as.numeric(ARIMAX_forecasting$mean))
```

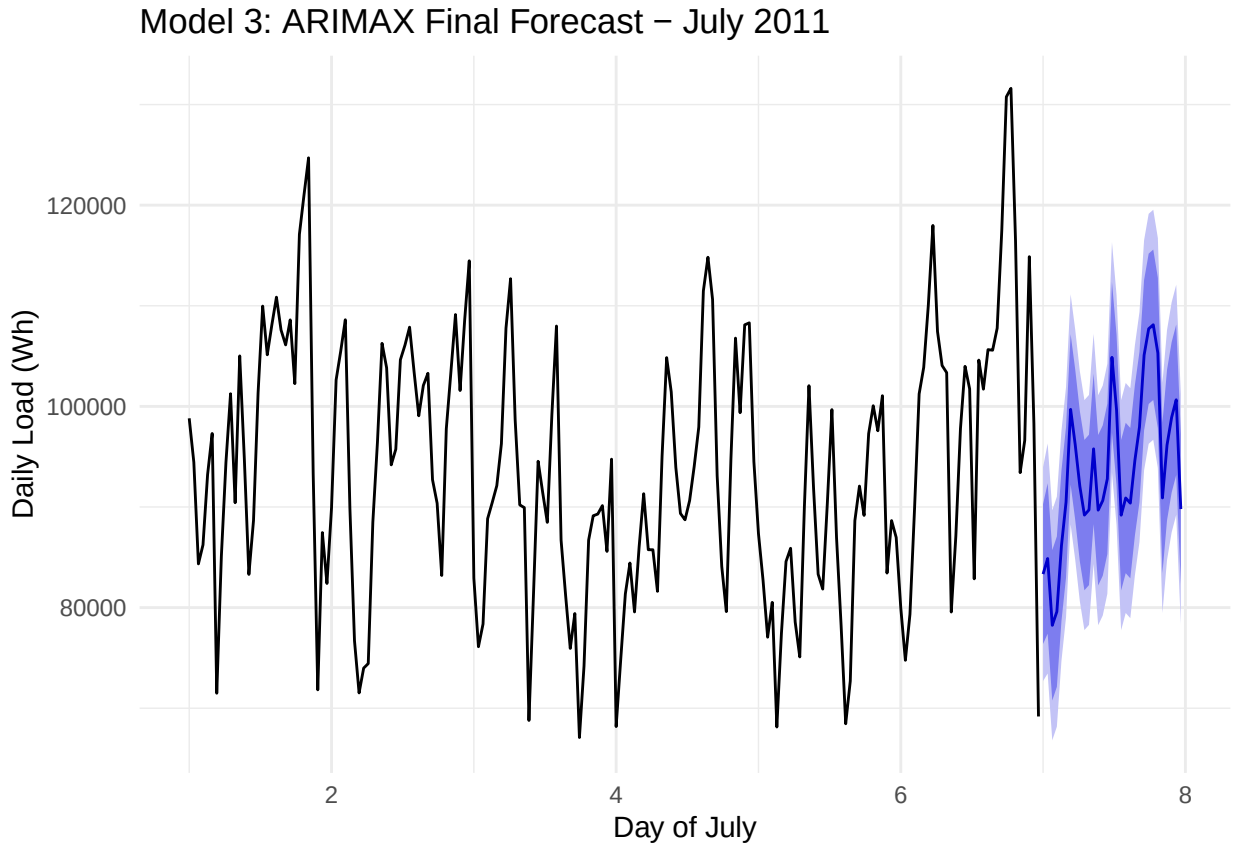
```
## [1] 5.497257
```

Final Forecast — July 2011

```
# Retrain ARIMAX on all July data (2005-2010)
ARIMAX_final <- auto.arima(ts_july_load, xreg = xreg_july,
                           seasonal = TRUE, stepwise = TRUE,
                           approximation = TRUE)
print(ARIMAX_final)
```

```
## Series: ts_july_load
## Regression with ARIMA(0,0,1)(1,0,0)[31] errors
##
## Coefficients:
##          ma1     sar1   temp_mean  temp_mean_sq  is_weekend
##       0.3949  -0.0618  -1275.0276       31.4807   3570.9929
## s.e.  0.0639   0.0849    162.0289        2.0599    969.1482
##
## sigma^2 = 29417456:  log likelihood = -1860.86
## AIC=3733.73   AICc=3734.19   BIC=3753.08
```

```r
# Forecast temperatures for July 2011 = average of 2009 and 2010
tmean_2009 <- july_df %>% filter(year(date) == 2009) %>% pull(temp_mean)
tmean_2010 <- july_df %>% filter(year(date) == 2010) %>% pull(temp_mean)
tmean_2011 <- (tmean_2009 + tmean_2010) / 2

# Weekend flags for July 2011
dates_july2011 <- seq.Date(as.Date("2011-07-01"),
                           as.Date("2011-07-31"), by = "day")
wknd_2011 <- as.numeric(wday(dates_july2011) %in% c(1, 7))

# xreg for forecast period
xreg_2011 <- cbind(
  temp_mean    = tmean_2011,
  temp_mean_sq = tmean_2011 ^ 2,
  is_weekend   = wknd_2011
)

# Generate forecast
ARIMAX_fc_2011 <- forecast(object = ARIMAX_final,
                           xreg = xreg_2011, h = 31)

autoplot(ARIMAX_fc_2011) +
  labs(title = "Model 3: ARIMAX Final Forecast - July 2011",
       x = "Day of July", y = "Daily Load (Wh)") +
  theme_minimal()
```




```r
# Build and save submission
submission_final <- data.frame(
  date = format(dates_july2011, "%Y-%m-%d"),
  load = round(as.numeric(ARIMAX_fc_2011$mean), 2)
)

print(submission_final)
```

```
##          date      load
## 1  2011-07-01  83330.41
## 2  2011-07-02  84879.88
## 3  2011-07-03  78238.23
## 4  2011-07-04  79596.95
## 5  2011-07-05  86140.30
## 6  2011-07-06  90481.83
## 7  2011-07-07  99704.11
## 8  2011-07-08  96194.46
## 9  2011-07-09  92069.75
## 10 2011-07-10  89193.98
## 11 2011-07-11  89712.12
## 12 2011-07-12  95778.45
## 13 2011-07-13  89681.77
## 14 2011-07-14  90641.16
## 15 2011-07-15  92804.27
## 16 2011-07-16 104876.88
## 17 2011-07-17  99653.01
```

```
## 18 2011-07-18  89183.90
## 19 2011-07-19  90896.64
## 20 2011-07-20  90382.93
## 21 2011-07-21  94638.61
## 22 2011-07-22  97952.45
## 23 2011-07-23 105089.92
## 24 2011-07-24 107712.56
## 25 2011-07-25 108112.09
## 26 2011-07-26 105302.21
## 27 2011-07-27  90898.23
## 28 2011-07-28  96157.91
## 29 2011-07-29  98908.55
## 30 2011-07-30 100657.08
## 31 2011-07-31  89793.55
```

```r
summary(submission_final$load)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   78238   89697   92070   93828   99281  108112
```

```r
write.csv(submission_final,
          "./output/submission_6thtry.csv",
          row.names = FALSE)
```

```r
# Make sure this says rowMeans not rowSums
load_processed <- raw_load_data %>%
  mutate(
    date       = as.Date(date),
    daily_load = rowMeans(across(starts_with("h")), na.rm = TRUE)
  ) %>%
  select(date, daily_load) %>%
  arrange(date)


# Must pass all three checks before uploading
cat("Row count:", nrow(submission_final), "\n")       # must be 31
```

```
## Row count: 31
```

```r
cat("First date:", submission_final$date[1], "\n")    # must be 2011-07-01
```

```
## First date: 2011-07-01
```

```r
cat("Last date:", submission_final$date[31], "\n")    # must be 2011-07-31
```

```
## Last date: 2011-07-31
```

```r
cat("Load range:", round(range(submission_final$load), 0), "\n")
```

```
## Load range: 78238 108112
```

```
# must be roughly 3000-5000
```

```
library(readxl)
library(lubridate)
library(forecast)
library(tidyverse)

# Load raw data
raw_load_data <- read_excel(
  "~/Documents/ENVIRON 797 - TSA Spring 2026/Forecasting2026/data/load.xlsx")

# Process with rowMeans
load_processed <- raw_load_data %>%
  mutate(
    date       = as.Date(date),
    daily_load = rowMeans(across(starts_with("h")), na.rm = TRUE)
  ) %>%
  select(date, daily_load) %>%
  arrange(date)

# Verify scale immediately
cat("Load range:", round(range(load_processed$daily_load), 0), "\n")
```

```
## Load range: 2247 7897
```

```
# Should show roughly 2000 to 8000
```